

Python 3.13.13 (tags/v3.13.13:01104ce, Apr 7 2026, 19:25:48) [MSC v.1944 64 bit (AMD64)] on win32

Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/C03.py =====

=====
A* SEARCH DRY RUN : START = A, GOAL = G
=====

Iteration 1
Current Node : A
Open List : ['A(g=0,h=7,f=7)']
Closed List : []
g(A) = 0, h(A) = 7, f(A) = 7

Iteration 2
Current Node : B
Open List : ['B(g=2,h=6,f=8)', 'C(g=4,h=4,f=8)']
Closed List : ['A']
g(B) = 2, h(B) = 6, f(B) = 8

Iteration 3
Current Node : E
Open List : ['E(g=4,h=2,f=6)', 'C(g=4,h=4,f=8)', 'D(g=9,h=3,f=12)']
Closed List : ['A', 'B']
g(E) = 4, h(E) = 2, f(E) = 6

Iteration 4
Current Node : G
Open List : ['G(g=6,h=0,f=6)', 'C(g=4,h=4,f=8)', 'D(g=6,h=3,f=9)', 'D(g=9,h=3,f=12)']
Closed List : ['A', 'B', 'E']
g(G) = 6, h(G) = 0, f(G) = 6

=====
GOAL REACHED!
Optimal Path : A -> B -> E -> G
Total Path Cost : 6
=====

===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/C03.py =====

=====
A* SEARCH DRY RUN : START = B, GOAL = D
=====

Iteration 1
Current Node : B
Open List : ['B(g=0,h=6,f=6)']
Closed List : []
g(B) = 0, h(B) = 6, f(B) = 6

Iteration 2
Current Node : E
Open List : ['E(g=2,h=2,f=4)', 'C(g=3,h=4,f=7)', 'A(g=2,h=7,f=9)',
'D(g=7,h=3,f=10)']
Closed List : ['B']
g(E) = 2, h(E) = 2, f(E) = 4

Iteration 3
Current Node : G
Open List : ['G(g=4,h=0,f=4)', 'C(g=3,h=4,f=7)', 'D(g=4,h=3,f=7)',
'A(g=2,h=7,f=9)', 'D(g=7,h=3,f=10)']
Closed List : ['B', 'E']
g(G) = 4, h(G) = 0, f(G) = 4

Iteration 4
Current Node : C
Open List : ['C(g=3,h=4,f=7)', 'D(g=4,h=3,f=7)', 'A(g=2,h=7,f=9)',
'D(g=7,h=3,f=10)']
Closed List : ['B', 'E', 'G']
g(C) = 3, h(C) = 4, f(C) = 7

Iteration 5
Current Node : D
Open List : ['D(g=4,h=3,f=7)', 'A(g=2,h=7,f=9)', 'D(g=7,h=3,f=10)']
Closed List : ['B', 'C', 'E', 'G']
g(D) = 4, h(D) = 3, f(D) = 7

=====
=====

GOAL REACHED!
Optimal Path : B -> E -> D
Total Path Cost : 4

=====
=====

>>>
===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/C03.py =====

=====
=====

MINIMAX WITH ALPHA-BETA PRUNING DRY RUN

=====
=====

Entering MAX node [MAX] (alpha=-inf, beta=inf)
Entering MIN node [MAX-1] (alpha=-inf, beta=inf)

```
Leaf MAX-1-1 = 3
-> After child MAX-1-1: value=3, alpha=-inf, beta=3
Leaf MAX-1-2 = 5
-> After child MAX-1-2: value=3, alpha=-inf, beta=3
Leaf MAX-1-3 = 6
-> After child MAX-1-3: value=3, alpha=-inf, beta=3
MIN node [MAX-1] selected value = 3
-> After child MAX-1: value=3, alpha=3, beta=inf
Entering MIN node [MAX-2] (alpha=3, beta=inf)
  Leaf MAX-2-1 = 9
  -> After child MAX-2-1: value=9, alpha=3, beta=9
  Leaf MAX-2-2 = 1
  -> After child MAX-2-2: value=1, alpha=3, beta=1
  ** PRUNING remaining children of MAX-2: [2] (beta <= alpha)
MIN node [MAX-2] selected value = 1
-> After child MAX-2: value=3, alpha=3, beta=inf
MAX node [MAX] selected value = 3
```

```
=====
=====
FINAL MINIMAX VALUE : 3
PRUNED NODES       : [('MAX-2', [2])]
=====
=====
```